Microservices

“Muốn đi nhanh và xa, cần thầy + đồng đội”



Outline

1. Microservice Boundary
2. Distributed Transaction

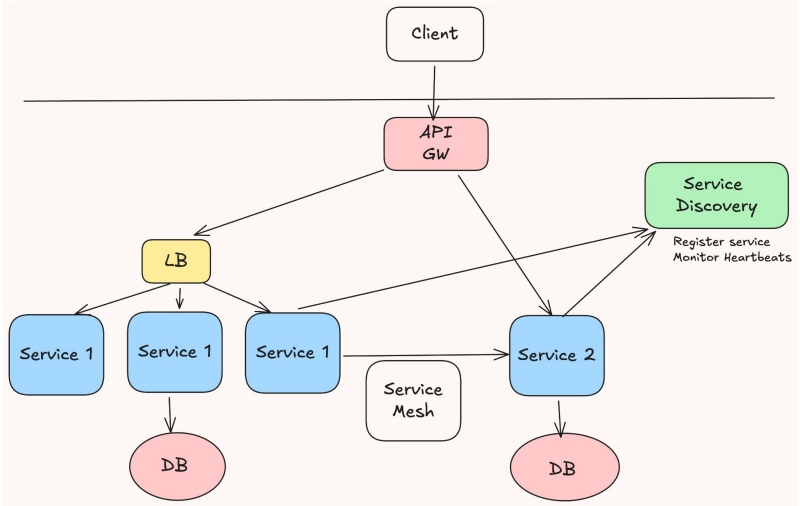
1. Fundamentals

Microservices

- Pros
  - Independent deployments
  - Organizational Alignment: Reduce communication costs
  - Scalability
  - Flexibility on technologies
  - Resilience
- Cons
  - Operation overhead: monitoring, troubleshooting, cost
  - Testing
  - Latency
  - Data Inconsistency

Components in Microservices

- Main Components
  - Microservices
  - Service Discovery
  - API Gateway
  - Load Balancer
- Extended Component
  - Message Broker
  - Monitoring System (logs, metrics)
  - Tracing System (APM)
  - Config/Secret Management System
  - Service Mesh
  - …
- Kubernetes support main components



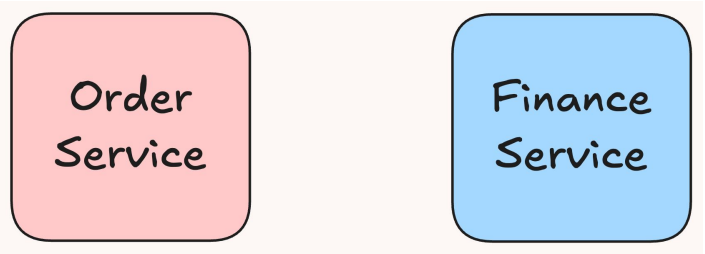
How to define microservice boundaries?

Process

- Using Event Storming + Domain Storytelling → discovery subdomains
- Using Bounded Context Canvas to define bounded contexts
- Use context map to define relationships and communication patterns between BCs
- **A good starting point: a Bounded Context → a microservice**
- **A Bounded Context might be large still** → continue to decompose into microservices

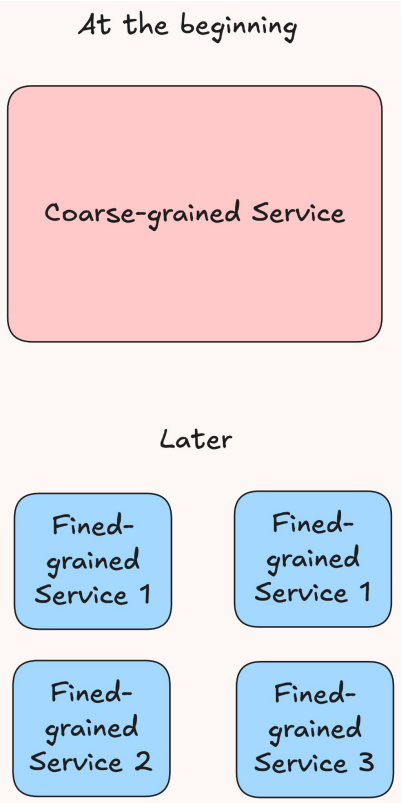
Principles/Techniques of Microservice Decomposition

- **High cohesion and low coupling**



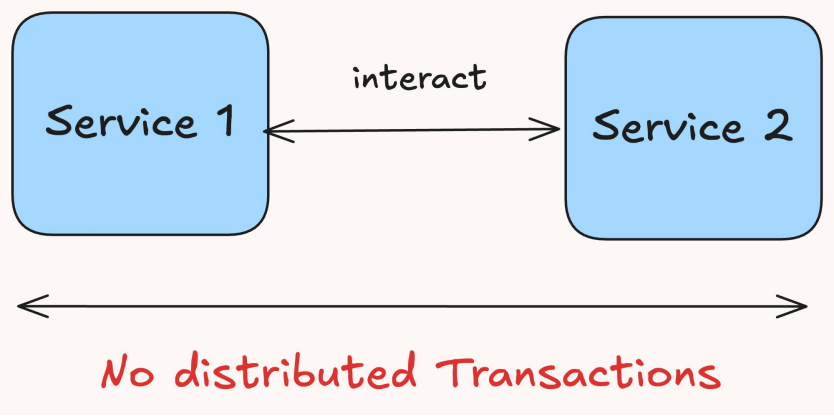
Principles/Techniques of Microservice Decomposition

- High cohesion and low coupling
- **First splitting into coarse-grained services, then splitting into fine-grained services.**



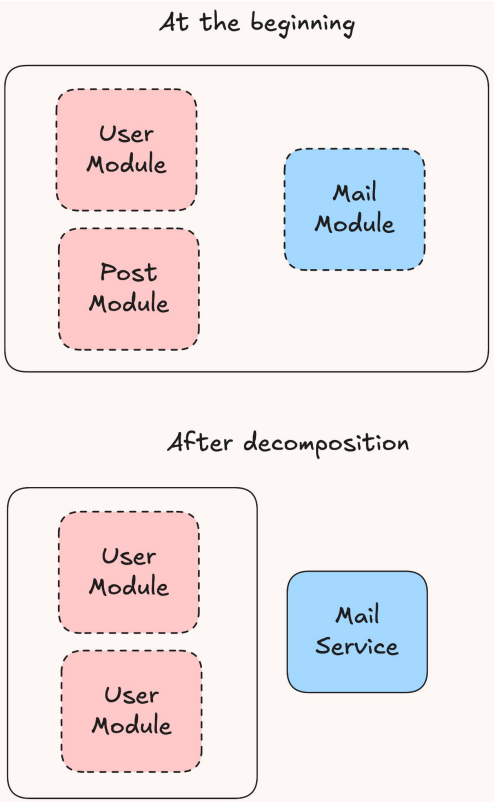
Principles/Techniques of Microservice Decomposition

- High cohesion and low coupling
- First splitting into coarse-grained services, then splitting into fine-grained services.
- **Avoid or limit the number of distributed transactions**



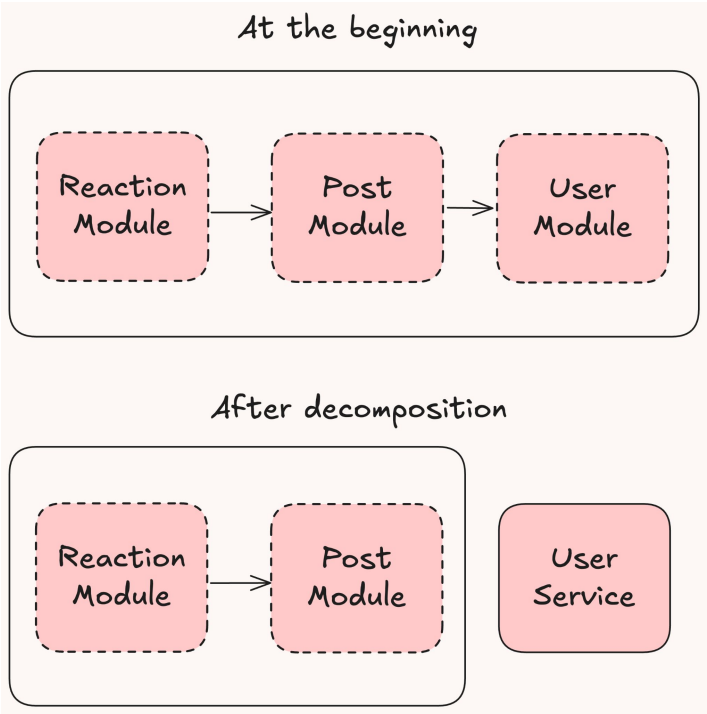
Principles/Techniques of Microservice Decomposition

- High cohesion and low coupling
- First splitting into coarse-grained services, then splitting into fine-grained services.
- Avoid or limit the number of distributed transactions
- Avoid affecting the daily deployment
    - **Start by splitting non-core services first → reduce impact.**



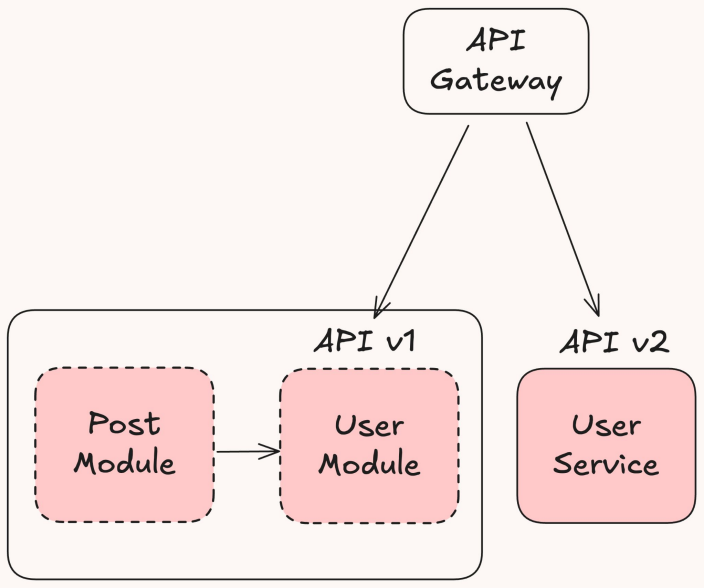
Principles/Techniques of Microservice Decomposition

- High cohesion and low coupling
- First splitting into coarse-grained services, then splitting into fine-grained services.
- Avoid or limit the number of distributed transactions
- Avoid affecting the daily deployment
    - Start by splitting non-core services first
      → reduce impact.
    - **Start by splitting the root service first**



Principles/Techniques of Microservice Decomposition

- High cohesion and low coupling
- First splitting into coarse-grained services, then splitting into fine-grained services.
- Avoid or limit the number of distributed transactions
- Avoid affecting the daily deployment
  - Start by splitting non-core services first → reduce impact.
  - Start by splitting the root service first
- **At the beginning of decomposition, keep both the module in the original service and a new microservice. Dual run the 2 components with different API versions.**



2. Distributed Transaction

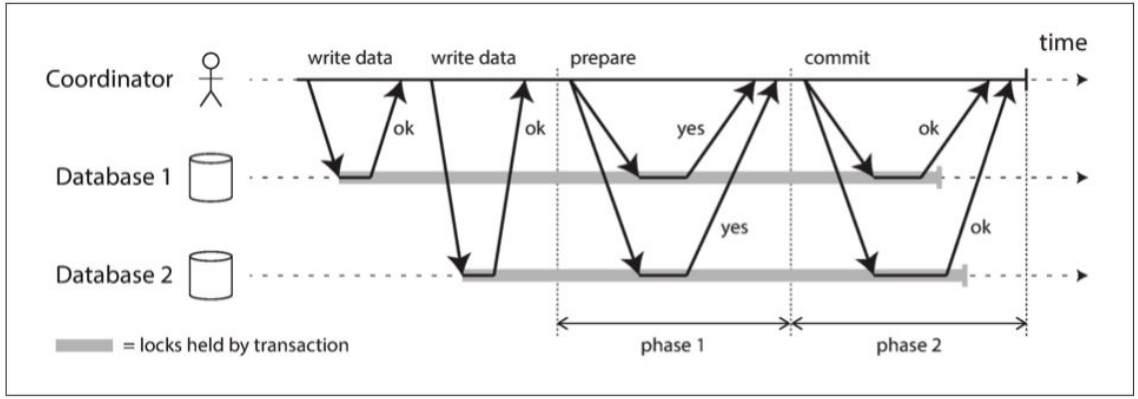
ACID in Transaction

- ACID
  - Atomicity: a group of SQL queries is like a query. all or nothing.
  - Consistency: Before and after the transaction, data should be valid according to all predefined rules.
  - Isolation: Multiple concurrent transactions do not affect each others.
  - Durability: Committed data would be not lost, even after power failure.
- The **main goal of transaction is atomicity**.
- **Multiple concurrent distributed transactions will violate the Isolation property.**
- Local transactions take care Consistency, Durability
  - Remind: Consistency in ACID != Consistency in CAP

Category

1. Internal Distributed Transaction
    - NewSQL
2. Cross-Database, Cross-Service
    - Discuss in this lecture

Two Phase Commit (2PC - XA)

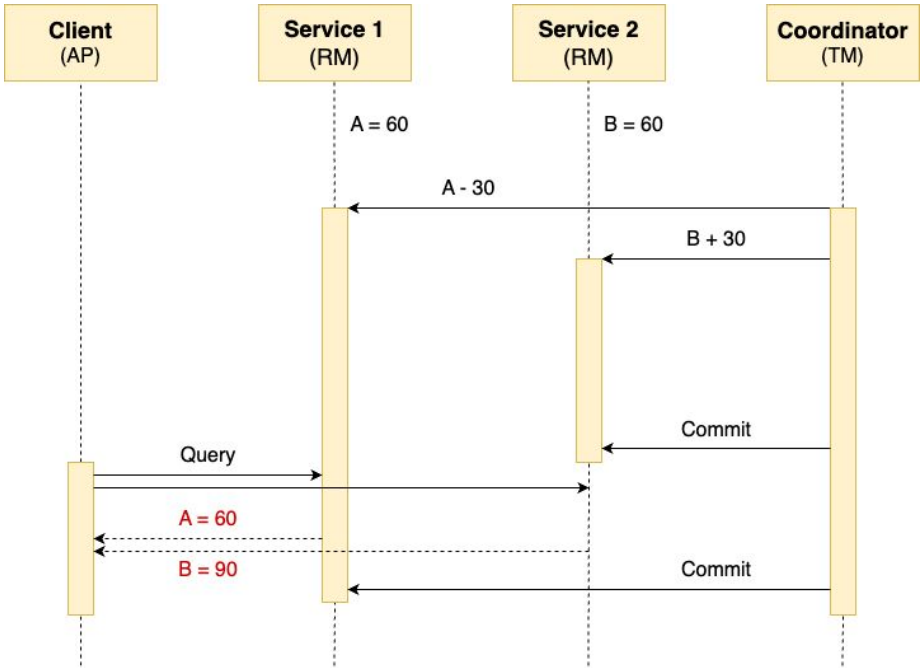


- Pros
  - Simple
  - **Strong Consistency*** for the happy case
- Cons
  - **Performance impact.** Long period of locking
  - Recovery failure when TM fails
  - Limited data source support

Strong Consistency in Theory

- Every read operation retrieves the most recent write
- All clients always have a consistent and up-to-date view of the data

Unable to Get Strong Consistency in Practice



*NewSQL DB is a distributed system that achieve strong consistency. Discuss later.

Consistency Level

- 2PC (XA) > AT > Try Confirm/Cancel (TCC) > 2 Phase Message > Saga

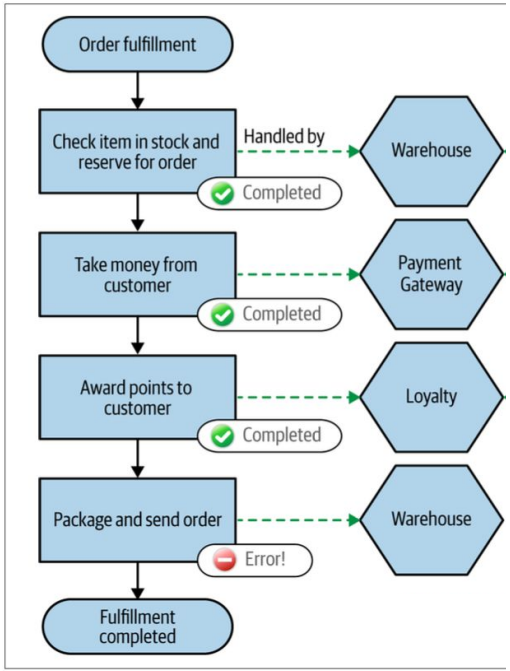
Why Saga is popular in Microservices?

Why Saga is popular in Microservices?

- **Reason 1: Eventual Consistency is Acceptable**
  - After creating an order successfully, users don’t care the inventory.
  - A little delay in a payment step (update txn, order) is acceptable.
- **Reason 2: More Scalable, Highly Concurrent**
  - Discuss later

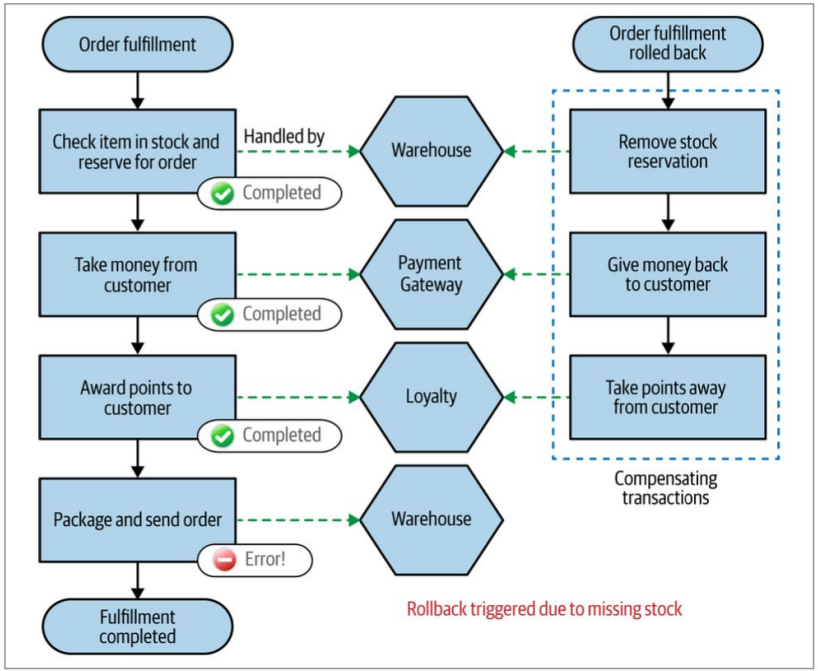
Saga

- Pros
  - No global lock + Non-blocking → improve concurrency
  - Partial failure recovery
- Cons
  - **Eventual consistency**
  - Complexity
  - Require idempotency and compensating operations



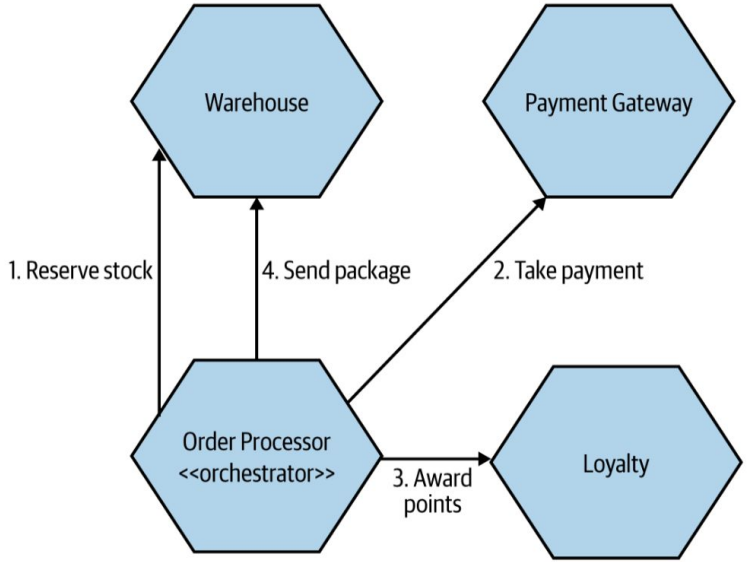
Saga

- Pros
  - No global lock + Non-blocking → improve concurrency
  - Partial failure recovery
- Cons
  - **Eventual consistency**
  - Complexity
  - Require idempotency and compensating operations



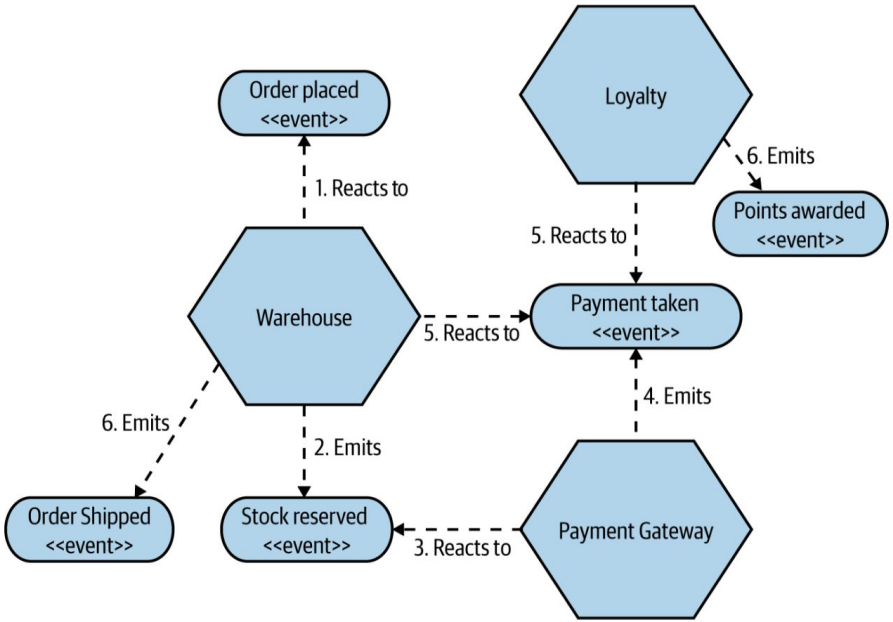
Saga / Orchestration

- Pros
  - Easy to debug
- Cons
  - Complexity
    - It’s hard to maintain a state machine
    - Centralizing too much business logic in the orchestrator



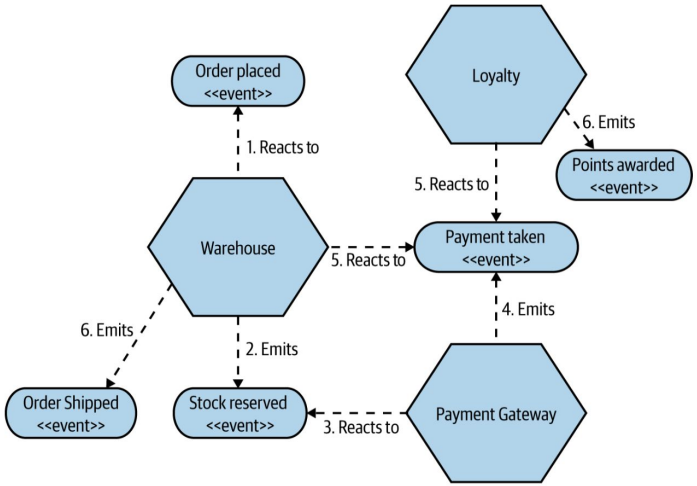
Sagas / Choreography

- Pros
  - No central TM
- Cons
  - Hard to debug.
    No global state for transactions

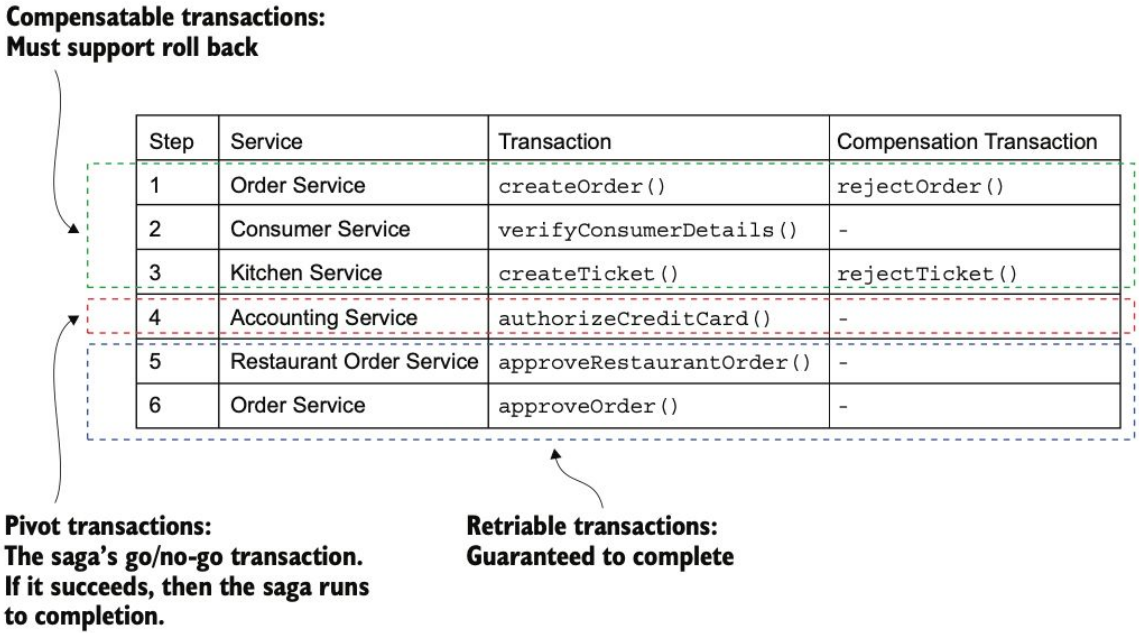


Choose which Saga?

- **My opinion: Choreography Saga**
- More scalable: Orchestration scalability depends TM's scalability
- Decoupling: Each team develop a service → only handling input event stream.
- More performant: More calls through TM in Orchestration
- More resilient: events can be stored in retention message broker.
- Orchestration is suitable to complex flows, but complex flow for a distributed transaction is a sign of anti-pattern.



Pivot Transaction



Step	Service	Transaction	Compensation Transaction
1	Order Service	`createOrder()`	`rejectOrder()`
2	Consumer Service	`verifyConsumerDetails()`	-
3	Kitchen Service	`createTicket()`	`rejectTicket()`
4	Accounting Service	`authorizeCreditCard()`	-
5	Restaurant Order Service	`approveRestaurantOrder()`	-
6	Order Service	`approveOrder()`	-

Saga Issues: The lack of Isolation

1. Lost Updates:
   - While `Creating Order` Saga is executing but `Canceling Order` Saga start
   - Solutions:
     - Sematic Lock.
       - Sematic Lock: Step 1 in `Creating Order` Saga changes state the order to APPROVAL_PENDING
       - Error Handling: `Canceling Order` Saga
         - Is blocked
         - Be failed → client retries (recommended)
     - Version file

Saga Issues: The lack of Isolation

2. Dirty Read:

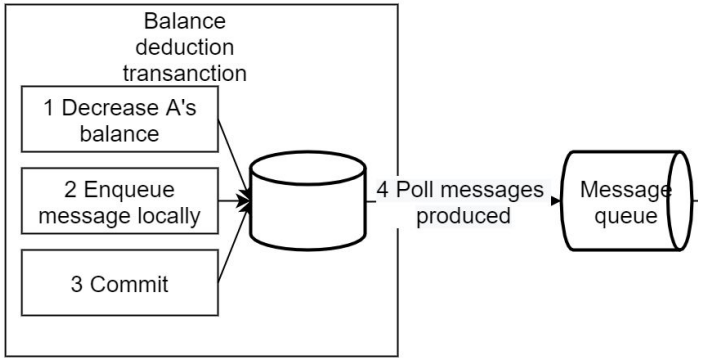
- Read committed data in a local transaction, but the global transaction is not committed yet.
- Solution:
  - Reorder
  - Re-read

Misunderstanding 1: Improving TPS

- **Not right. It depends on**
  - Architecture
  - Lock contention
- The case that distributed transaction won’t improve TPS:
  - High lock contention
- The case that distributed transaction will improve TPS:
  - The dataset is sharded or separated, and
  - Low lock contention.

Misunderstanding 2: Purpose of Each Pattern

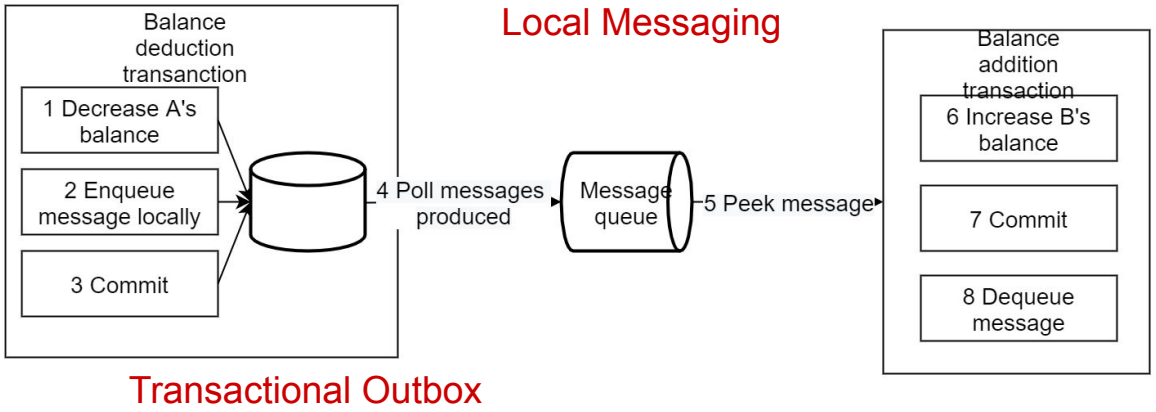
- Misunderstanding: Event Sourcing, Transactional Outbox solve distributed transaction
  - **Event Sourcing**:
    - Providing other persistence method than the traditional method.
    - Providing time machine :)
  - Transactional Outbox: Dual-write



Transactional Outbox

Misunderstanding 2: Purpose of Each Pattern

- Misunderstanding: Event Sourcing, Transactional Outbox solve distributed transaction
  - Event Sourcing:
    - Providing other persistence method than the traditional method.
    - Providing time machine :)
  - **Transactional Outbox: Dual-write**



Practices

Liveness

Liveness System

- Context:
  - 1M IoT devices send heartbeats to server periodically (every 3 seconds).
- Functional Requirement:
  - If a device does not send any heartbeat after a timeout threshold
    → System raise alert that the device is connected.
  - Ability to configure the timeout threshold for each device type.
- Non-functional Requirement:
  - Scalability
  - A little delay between the timeout moment and the alert moment is acceptable

Recap

- Microservice Boundary: Starting with Bounded Context → a Microservice → Decomposition later
- In microservices, eventual consistency is acceptable. So implement a distributed transaction by Choreography Saga to achieve high concurrency
- The main goal of Distributed Transaction is not improving TPS. In some case with separated datasets + low lock contention, distributed transaction will improve TPS.

Homework

- What are 4 conditions that causes deadlock.
- A simple way to avoid deadlock.
- Design Liveness System

References

- https://en.dtm.pub/
- https://seata.apache.org/docs/overview/what-is-seata

Thank you 🙏

